

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 05 · LECTURE NOTES

Hashing and Amortized Analysis

Constant time on average, and what 'average' is doing there.

MODULE OUTCOMES

- Explain collision resolution strategies and their trade-offs.
- Explain why universal hashing is needed and what it guarantees.
- Perform amortised analysis by aggregate, accounting, and potential methods.
- Prove dynamic array doubling is $O(1)$ amortised.
- Explain why hash tables degrade in practice and how to detect it.

1 From direct addressing to hashing

If keys are integers in a small range, a lookup is trivially $O(1)$: index an array. The difficulty is that real key spaces are enormous — all strings, all 64-bit integers — while the number of keys actually stored is small.

A **hash function** $h: U \rightarrow \{0, \dots, m-1\}$ compresses the key space to a table of m slots. Because $|U| > m$, the pigeonhole principle guarantees **collisions**: distinct keys mapping to the same slot. A hash table is therefore a direct-address table plus a policy for what to do when two keys want the same slot.

1.1 Chaining versus open addressing

	Chaining	Open addressing
Structure	Each slot points to a list of entries	All entries live in the array; collisions probe elsewhere
Load factor	α may exceed 1; degrades linearly	Requires $\alpha < 1$; degrades sharply as $\alpha \rightarrow 1$
Expected probes	$1 + \alpha$	$\sim (1 + 1/(1-\alpha)^2)/2$ for linear probing
Cache behaviour	Poor — each lookup chases a pointer to an unpredictable address	Excellent — probes are contiguous and prefetch well
Deletion	Trivial: unlink from the list	Needs tombstones, which accumulate and lengthen probes until a rehash
Used by	Textbooks; older standard libraries	Most modern high-performance implementations

THE CACHE ARGUMENT HAS CHANGED THE DEFAULT

Classical analysis favours chaining: the bounds are cleaner and it handles $\alpha > 1$. On modern hardware, open addressing usually wins, because a linear probe sequence is contiguous memory the prefetcher handles perfectly, whereas every chained lookup is a dependent load to an unpredictable address.

This is Module 01's theme again: two structures with the same asymptotic behaviour, separated by a large constant factor the model cannot see. CSCE 614 Module 05 explains the mechanism.

1.2 The load factor

$$\alpha = n / m$$

For chaining, the expected number of elements examined is $1 + \alpha$ — well-behaved, and doubling α roughly doubles the work. For linear probing the expected probe count is approximately $(1 + 1/(1-\alpha)^2)/2$, which is about 2.5 at $\alpha = 0.5$, about 8.5 at $\alpha = 0.75$, and about 50 at $\alpha = 0.9$.

That is not a gradual degradation, it is a cliff, and it is why open-addressed tables resize at a load factor around 0.7 rather than waiting until full. A system sized to run at 90% capacity will appear fine in testing and collapse in production.

2 Universal hashing

The $O(1 + \alpha)$ analysis assumes keys distribute uniformly across slots. For any *fixed* hash function that assumption is false for some input: with $|U|$ keys and m slots, some slot must receive at least $|U|/m$ of them.

THIS BECAME A REAL VULNERABILITY

Web frameworks place form parameters, query strings, and JSON keys directly into hash tables. An attacker who knows the hash function can submit a request whose parameters all collide, turning every table operation into a linear scan.

The resulting denial-of-service attacks affected most major web frameworks simultaneously around 2011. The deployed fix, essentially everywhere, was randomised hashing seeded per process.

A theoretical worst case became an operational emergency, which is worth remembering when a worst case looks too unlikely to care about.

2.1 The definition and what it buys

$$H \text{ is universal} \Leftrightarrow \text{for all } x \neq y: \Pr_{h \in H}[h(x) = h(y)] \leq 1/m$$

Choose h uniformly at random from H when the table is created. The expected chain length is then $O(1 + \alpha)$ for **every** input, with no distributional assumption — because the randomness is in the choice of h , not in the data.

This is precisely the move from Module 03. The bad case still exists, but it now depends on which hash function you happened to draw, which the adversary cannot observe. Average-case becomes expected-case, and the guarantee becomes unconditional.

3 Amortised analysis

Some operations are usually cheap and occasionally expensive. Amortised analysis bounds the cost of a *sequence*, showing that the expensive cases cannot occur often enough to matter.

NOT AVERAGE CASE

Average-case analysis is probabilistic and depends on an assumption about the input distribution.

Amortised analysis involves no probability whatsoever. It is a worst-case claim about a sequence of operations: *any* n operations cost at most X in total, always, including under adversarial ordering. The proof is a counting argument.

Conflating the two is one of the most common misunderstandings in this subject.

3.1 Three methods

Method	Technique	When to use
Aggregate	Bound the total cost of n operations directly, then divide by n .	Simple structures; always try this first.
Accounting	Assign each operation an amortised charge; overcharged cheap operations bank credit that pays for later expensive ones. Credit must never go negative.	When you can identify concretely which cheap operations pay for which expensive one.
Potential	Define Φ mapping the structure's state to a non-negative number. Amortised cost = actual cost + $\Delta\Phi$.	The general method. Use it for splay trees, Fibonacci heaps, and anything where the other two become awkward.

3.2 Dynamic arrays, done three ways

A growable array appends in $O(1)$ until full, then allocates double the capacity and copies everything: $O(n)$ for that one operation.

AGGREGATE

Over n appends, resizes happen at capacities $1, 2, 4, \dots, n$.
 Total elements copied = $1 + 2 + 4 + \dots + n < 2n$.
 Total cost $< n$ (appends) + $2n$ (copies) = $3n$.
 Amortised cost per append = $3 = O(1)$.

ACCOUNTING

Charge 3 credits per append: 1 pays the insert, 2 go in the bank.
 When a table of size n doubles, the $n/2$ elements inserted since the previous resize hold 2 credits each $\rightarrow n$ credits available, exactly covering the n copies. The balance never goes negative.

POTENTIAL

$\Phi = 2n - m$ (n = elements, m = capacity); $\Phi \geq 0$ since $m \leq 2n$.
 Ordinary append: actual = 1, $d\Phi = +2$ $\rightarrow$ amortised 3
 Resizing append: actual = n , $d\Phi = -(n-2)$ $\rightarrow$ amortised 3

GROWTH MUST BE MULTIPLICATIVE

Doubling: copies total $1 + 2 + 4 + \dots + n < 2n$. A geometric series converges, so the amortised cost is $O(1)$.

Growing by a fixed c : resizes at $c, 2c, 3c, \dots, n$, so copies total $c + 2c + \dots + n = \Theta(n^2/c)$. An arithmetic series does not converge, and the amortised cost per append is $\Theta(n)$.

This is why every growable buffer in every standard library grows by a factor (2, or 1.5 to allow memory reuse) rather than by an increment, and why calling `realloc` with `size+1` in a loop is a classic quadratic-blowup bug.

4 Why real hash tables disappoint

Cause	Symptom	Detection and fix
Poor hash function	Clustering; some probe sequences very long	Plot the probe-length distribution. Mix bits before masking; avoid using raw low bits of pointers or sequential IDs.
Load factor too high	Sudden collapse rather than gradual slowdown	Resize at $\alpha \approx 0.7$ for open addressing. Monitor α .
Tombstone accumulation	Degradation under delete-heavy workloads even at low α	Count tombstones; rehash when they exceed a threshold.
Cache misses	Throughput far below the $O(1)$ expectation	Prefer open addressing; store small values inline rather than behind a pointer.
Resize pauses	Occasional multi-millisecond latency spikes	Amortised $O(1)$ is a throughput claim. Under a latency budget, use incremental resizing that moves a few entries per operation.

MEASURE THE TAIL, NOT THE MEAN

Average lookup time hides exactly the behaviour that causes incidents. A table with a 2.1 average probe count and a 95th percentile of 60 is about to become someone's outage.

Instrument the probe-length distribution. It tells you about hash quality, load factor, and tombstones all at once, and it degrades visibly well before the average does.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.006 — Hashing with Chaining, Table Doubling, Amortization	Both halves of this module, carefully done, including the potential method.
MIT 6.046J — Amortized Analysis	The graduate treatment, with the potential method applied to more demanding structures.
Jeff Erickson, Algorithms — Hash Tables chapter	Universal hashing done properly, with the construction and the proof.
Google — Swiss Tables design notes (Abseil)	How a state-of-the-art open-addressed hash table is actually built, and why it is laid out the way it is. Short and worth reading.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement hash tables with chaining and with linear probing. Measure average and 99th-percentile probe count as α goes from 0.1 to 0.95, and plot both. Identify the cliff.
- 2 Implement a deliberately poor hash function (for example, the low 8 bits of the key) and measure the probe-length distribution against a good one.
- 3 Construct an adversarial key set for a fixed hash function in your language of choice, insert it, and measure the degradation. Then enable or simulate randomised seeding and repeat.
- 4 Implement a dynamic array with doubling and one that grows by +1. Measure total time for 10^6 appends and confirm the second is quadratic.
- 5 Prove by the potential method that a counter supporting increment has $O(1)$ amortised bit-flips. Choose Φ = number of 1 bits.
- 6 Instrument a delete-heavy workload on your open-addressed table and show tombstone accumulation degrading lookups. Implement rehash-on-threshold and show the recovery.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why are collisions unavoidable, and what are the two main strategies for handling them?
- 2 Why does open addressing degrade sharply near $\alpha = 0.7$ while chaining does not?
- 3 Why does a fixed hash function always have a bad input set, and what does universal hashing change?
- 4 State precisely how amortised analysis differs from average-case analysis.
- 5 Give the aggregate argument for dynamic array doubling, and explain why growing by a constant increment fails.
- 6 Amortised $O(1)$ is a claim about what, exactly? Give a situation where it is not the guarantee you need.